

Title Page

Title: GT Store

Author: Barnabe Marty, Padraig Littlefield

Date: November 29, 2025

Abstract

This project implements GT store, a distributed key-value store system, using network calls and simulating the shopping processes. The system implements a centralized manager architecture that coordinates a multitude of storage nodes, allowing data partitioning and replication failure mitigation.

This specific implementation uses gRPC for network communication and provides a simple API for client put and get operations. Failure detection mechanisms for storage node failure also exist to ensure a minimum fault tolerance through heartbeat monitoring, dynamic node updates, and data replication/redistribution. Storage nodes maintain communication with the manager for coordination, while clients seamlessly connect and handle node failures by refreshing the node registry and retrying.

Table of Contents

Title Page	1
Abstract	1
System Components.....	2
Centralized Manager	2
Storage	2
Client API	2
Client Driver Application	3
Design	4
Data Partitioning	4
Data Replication.....	4
Data Consistency	4
Design Tradeoffs	5
Performance Analysis	6
Basic Test Cases	6
Throughput Tests.....	6
Load Balance Test	7
Appendix A	8

System Components

Centralized Manager

The manager is the central/focal point of the GTStore system. The manager ensures consistency through the system by keeping track of all storage nodes and ensuring data redistribution/replication in case of failure, as well as being a point of reference for the client when it comes to accessing storage node metadata.

This manager runs as a standalone gRPC server process and keeps track of all storage nodes by registering them with the system upon request, assigning them a node identifier, and monitoring node availability through periodic heartbeat messages (every 5 seconds, with 15 second timeout to determine node failure). Consistent hashing is also put into place to compute primary and replica node assignments, ensuring balanced distribution of data across active nodes.

Upon detecting failure of a node (through a missed heartbeat), data redistribution gets triggered, and the manager instructs storage nodes to replicate data to new replica locations, maintaining configured replication factor (if possible, i.e. there are still enough storage nodes available for the corresponding replica factor)

Storage

Storage nodes are independent gRPC server processes that maintain the data (with in-memory hash maps) for actual key-value storage. Upon startup (provided that the manager IP address and port is given to establish a connection), the storage node sends its IP address and port over to the manager, which then returns it a node identifier. After successful registration, the node creates a new child thread to begin the periodic heartbeat transmissions over to the manager.

Nodes expose four RPC services: `Put` for storing key-value pairs, `Get` for retrieving values, `GetAllData` for returning their entire data store (especially useful in redistribution), and `ReplicateData` for bulk data replication. All operations are performed with thread safe mutexes for protection. Storage nodes are independent and do not communicate with each other (only to the manager and client), simplifying the whole system.

Client API

The client library API enables any user to use the system via these four functions: `init()` to establish a connection with the manager and fetches the current node registry/map, `put(key, value)` allows for a key-value pair to be sent and stored in corresponding

storage nodes, `get(key)` retrieves the key-value pair from storage nodes by trying replicas in order until success, and `finalize()` cleans up the current client object and disconnects from the manager.

When initializing, the client keeps track of all storage nodes via a node map which it acquires from the manager via gRPC calls from the `refresh_node_registry()` function. In case, of a failed put or get call from a timeout (network failure, i.e. storage node failure), the client calls `refresh_node_registry()` once more and tries again with the new updated node registry it now has. To ensure replication, when a key is put into a storage node, this is repeated several times to multiple storage nodes, as to fulfill the replication requirement necessary for data consistency.

Client Driver Application

The designed client application uses the API for easy interaction with the server but also in various ways. The client application can be used in two distinct approaches: one time put or get value to the storage node, or an interactive “shell” that can be used to continuously take input from a user to put and get key-value pairs. In both cases, the client driver application can take a custom manager’s address but assumes 127.0.0.1:50051 as the default (made testing much easier). Alongside that, the client is always initialized with client identifier 1 for ease of use and testability.

Design

Data Partitioning

The system is implemented with consistent hashing with active node filtering for data partitioning. Each key is hashed (use C++ standard hash) to make an integer, which is then mapped to storage nodes via the modulo operation over the sorted list of active nodes. This approach is extremely advantageous because it uniformly distributes your key-value pairs (as shown in our load balance test in the Performance analysis) and is deterministic by nature, meaning that the client, storage, and manager can perform this sort of arithmetic and get the same result for key-value pair mapping.

A huge bottleneck, however, is that redistribution requires a lot of computation because the manager node must rehash affected keys. So, on node failures the system does throttle a bit to remap/rehash all key value pairs, but for our experiments with a small number of nodes and key-value pairs this is not as much of a problem.

Data Replication

Data replication requires us to first store a key-value pair in one storage node (like how we would normally for data partitioning) and then store K-1 additional replicas are stored in consecutive active nodes (wrapping around circularly). Replication happens synchronously at write time (when the client calls `put()`, it directly writes to all K replica nodes via separate gRPC calls), and upon node failure, the manager triggers data redistribution and contacts all surviving nodes, retrieving their data, and recalculating missing replicas to then push them into appropriate nodes using `put` gRPC call.

Data Consistency

GTStore enables immediate data consistency because write operations block until data is successfully stored on most replica nodes, ensuring that committed writes are persistent and durable across multiple nodes before acknowledgement. Read operations also enable consistency by attempting to read all replicas in order and returning the first successful response.

In the case of node failures, the manager is made aware with a heartbeat mechanism and accordingly redistributes all data node replication to ensure strong consistency across the system.

Design Tradeoffs

GTStore prioritizes strong consistency and operational simplicity at the cost of write performance. First, by implementing synchronous replication where write block until majority acknowledgement from all K replicas (i.e. the client writes the replicas), GTStore guarantees extremely strong consistency and durable data. However, this design decision significantly reduces system performance, especially when increasing the replication factor (check performance analysis section for more information).

Additionally, another tradeoff is write latency due to the necessity for high computation (hashing and modulo) to map key-value pairs to storage nodes. This is especially important during data redistribution because, even though the system has great load balancing, there is a small bottleneck since a lot of key-value pairs must be computed and stored back into nodes. A tradeoff that we are willing to make for good load balancing.

Performance Analysis

Basic Test Cases

To view Test 1 through 4 outputs, you can simply run `./test#.sh` or review Appendix A for screenshots of the output of each test case.

Throughput Tests

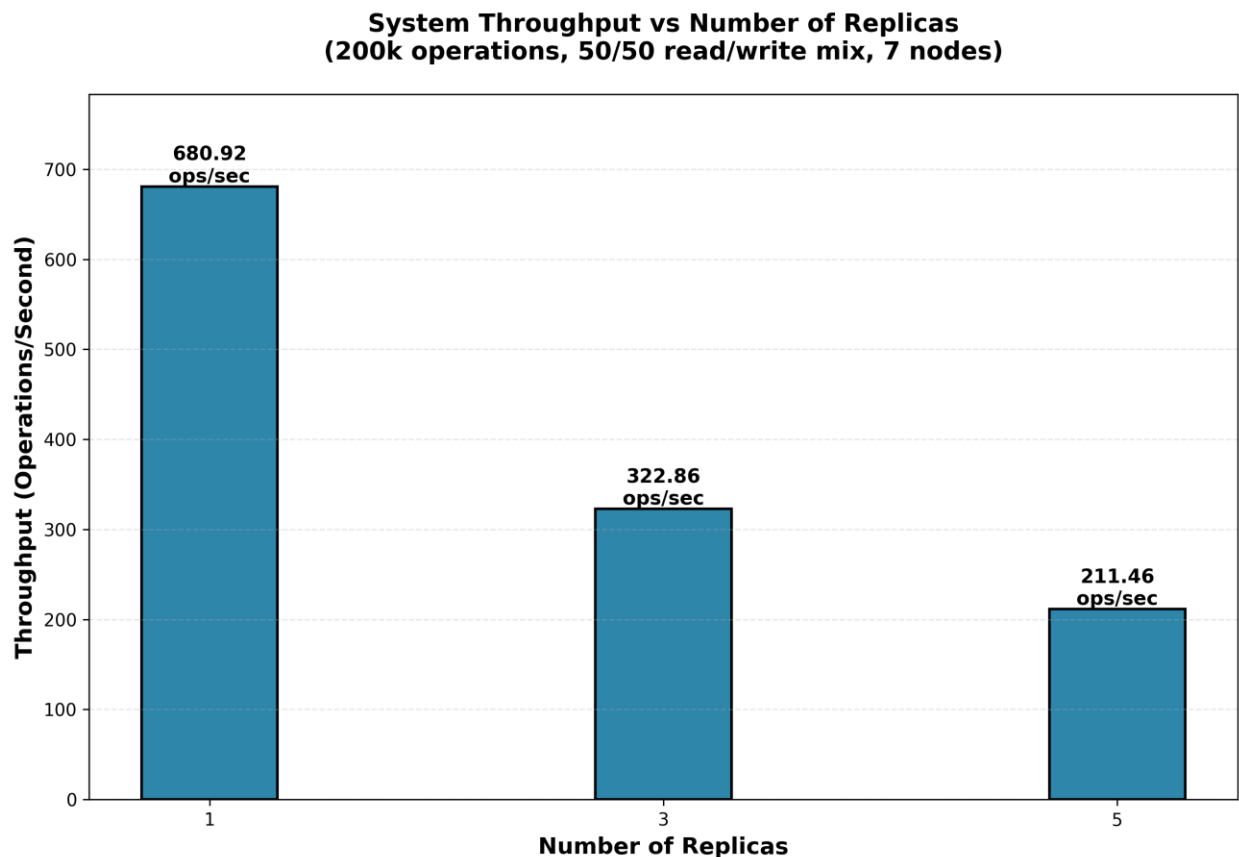


Figure 1: GTStore Throughput Test over various Replication Factors

The analysis of the throughput test highlight the expected tradeoffs between the replication factor and the performance of distributed storage systems. The single replica configuration achieved the highest throughput, with 680.92 operations per second. Increasing the replication factor to 3 reduced throughput to 322.86 ops/sec, marking a 52.6% reduction in throughput. Further increasing the replication factor continues to reduce the throughput, but at a diminishing rate, as 5 replicas reduced performance by 34.5%, to 211.46 ops/sec.

Load Balance Test

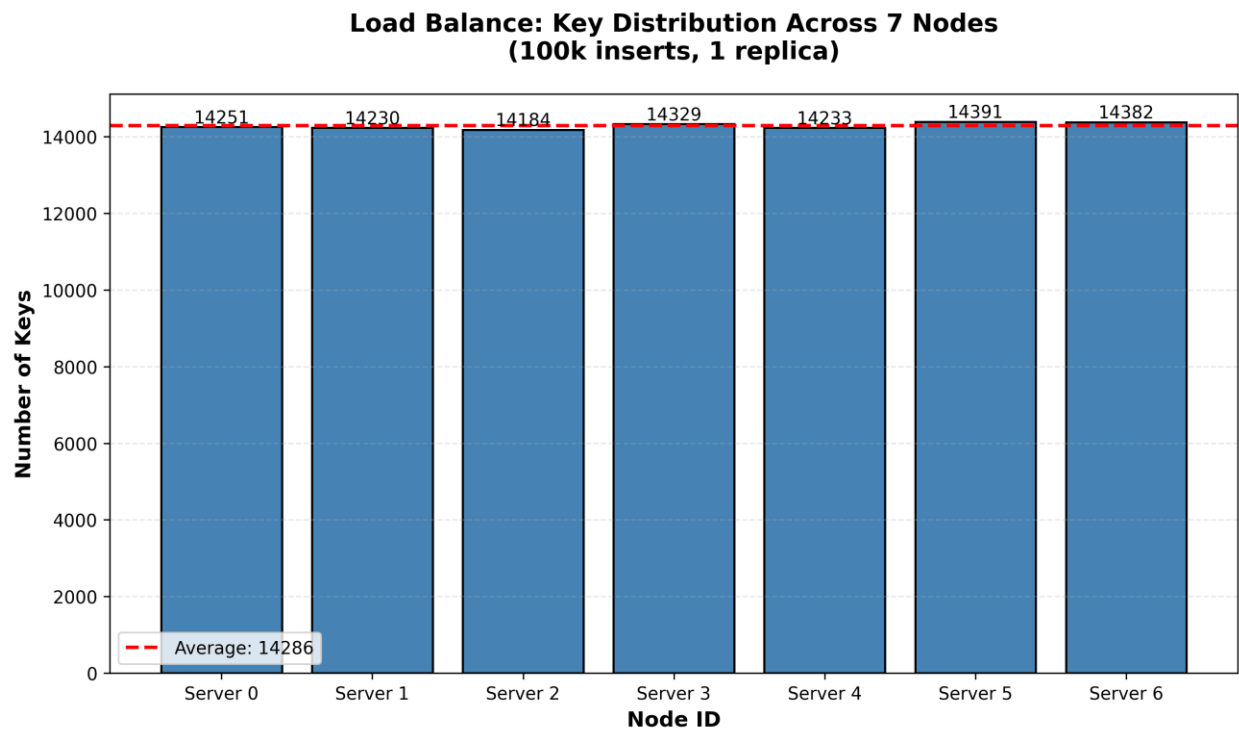


Figure 2: GTStore Memory Load Spread over Storage Nodes (Replication Factor = 1)

The load balance test demonstrates that our distribution mechanism manages to achieve excellent uniformity across the system. With 100,000 key insertions across 7 nodes, for a perfect system, we should expect that each node contains 14,286 keys. Our system had a range of 14,184 to 14,391 keys/node, representing a total variance of 1.5% from the ideal, with a standard deviation of only 75.2 keys. This distribution indicates that our hashing function effectively distributes the load, preventing hotspots and bottlenecks.

Appendix A

Test 1:

```
=== Test 1: Basic Single Server GET/PUT ===

>>> PUT key1 = value1
OK, server0

>>> GET key1 (expect: key1, value1, server...)
key1, value1, server0

>>> PUT key1 = value2 (overwrite)
OK, server0

>>> PUT key2 = value3 (expect: key2, value3, server...)
OK, server0

>>> PUT key3 = value4 (expect: key3, value4, server...)
OK, server0

>>> GET key1 (expect: key1, value2, server...)
key1, value2, server0

>>> GET key2 (expect: key2, value3, server...)
key2, value3, server0

>>> GET key3 (expect: key3, value4, server...)
key3, value4, server0

=== Test 1 Complete ===
```

Test 2:

```
=== Test 2: Basic Multi-Server GET/PUT ===

>>> PUT key1 = value1
OK, server2

>>> GET key1 (expect: key1, value1, server...)
key1, value1, server2

>>> PUT key1 = value2 (overwrite)
OK, server2

>>> PUT key2 = value3 (expect: key2, value3, server...)
OK, server1

>>> PUT key3 = value4 (expect: key3, value4, server...)
OK, server3

>>> GET key1 (expect: key1, value2, server...)
key1, value2, server2

>>> GET key2 (expect: key2, value3, server...)
key2, value3, server1

>>> GET key3 (expect: key3, value4, server...)
key3, value4, server3

=== Test 2 Complete ===
```


Test 3:

```
=== Test 3: Availability thorough Single Node Failure ===

>>> PUT key1 = value1
OK, server1
>>> PUT key2 = value2
OK, server1
>>> PUT key3 = value3
OK, server0
>>> PUT key4 = value4
OK, server2
>>> PUT key5 = value5
OK, server2
>>> PUT key6 = value6
OK, server0
>>> PUT key7 = value7
OK, server1
>>> PUT key8 = value8
OK, server1
>>> PUT key9 = value9
OK, server1
>>> PUT key10 = value10
OK, server0
>>> PUT key11 = value11
OK, server1
>>> PUT key12 = value12
OK, server0
>>> PUT key13 = value13
OK, server2
>>> PUT key14 = value14
OK, server2
>>> PUT key15 = value15
OK, server0
Manually kill storage nodes via kill -9 <PID>
gtstore.pids contains the PIDs of the running nodes. (Ex: storageX <PID> <PORT>)
Press Enter once you have killed 1 storage node...

Waiting 20 seconds for system to stabilize...
Node 1 missed heartbeat, marking as failed
Triggering data redistribution due to failure of node 1
```

```
>>> GET key1 (expect: key1, value1, server...)
key1, value1, server0

>>> GET key2 (expect: key2, value2, server...)
key2, value2, server0

>>> GET key3 (expect: key3, value3, server...)
key3, value3, server0

>>> GET key4 (expect: key4, value4, server...)
key4, value4, server0

>>> GET key5 (expect: key5, value5, server...)
key5, value5, server0

>>> GET key6 (expect: key6, value6, server...)
key6, value6, server2

>>> GET key7 (expect: key7, value7, server...)
key7, value7, server2

>>> GET key8 (expect: key8, value8, server...)
key8, value8, server0

>>> GET key9 (expect: key9, value9, server...)
key9, value9, server2

>>> GET key10 (expect: key10, value10, server...)
key10, value10, server2

>>> GET key11 (expect: key11, value11, server...)
key11, value11, server2

>>> GET key12 (expect: key12, value12, server...)
key12, value12, server0

>>> GET key13 (expect: key13, value13, server...)
key13, value13, server2

>>> GET key14 (expect: key14, value14, server...)
key14, value14, server0

>>> GET key15 (expect: key15, value15, server...)
key15, value15, server2
```

Test 4:

```
=== Test 4: Availability through multiple node failures ===

Inserting 20 key-value pairs...
>>> PUT key1 = value1
OK, server2
>>> PUT key2 = value2
OK, server2
>>> PUT key3 = value3
OK, server4
>>> PUT key4 = value4
OK, server4
>>> PUT key5 = value5
OK, server0
>>> PUT key6 = value6
OK, server1
>>> PUT key7 = value7
OK, server4
>>> PUT key8 = value8
OK, server1
>>> PUT key9 = value9
OK, server4
>>> PUT key10 = value10
OK, server1
>>> PUT key11 = value11
OK, server6
>>> PUT key12 = value12
OK, server1
>>> PUT key13 = value13
OK, server6
>>> PUT key14 = value14
OK, server6
>>> PUT key15 = value15
OK, server0
>>> PUT key16 = value16
OK, server0
>>> PUT key17 = value17
OK, server2
>>> PUT key18 = value18
OK, server3
>>> PUT key19 = value19
OK, server1
>>> PUT key20 = value20
OK, server6

Overwriting keys 5, 10, 15...
>>> PUT key5 = newvalue5
OK, server0
>>> PUT key10 = newvalue10
OK, server1
>>> PUT key15 = newvalue15
OK, server0
Manually kill storage nodes via kill -9 <PID>
gtstore.pids contains the PIDs of the running nodes. (Ex: storageX <PID> <PORT>)
Try to kill nodes that hold keys 5, 10, or 15 if possible.
Press Enter once you have killed 2 storage nodes...

Waiting 20 seconds for system to stabilize...
Node 0 missed heartbeat, marking as failed
Triggering data redistribution due to failure of node 0
Failed to get data from node 2: failed to connect to all addresses; last error: l
Node 2 missed heartbeat, marking as failed
Triggering data redistribution due to failure of node 2

>>> GET key1 (expect: key1, value1, server...)
key1, value1, server4
>>> GET key2 (expect: key2, value2, server...)
key2, value2, server3
>>> GET key3 (expect: key3, value3, server...)
key3, value3, server5
>>> GET key4 (expect: key4, value4, server...)
key4, value4, server5
>>> GET key5 (expect: key5, value5, server...)
key5, newvalue5, server3
>>> GET key6 (expect: key6, value6, server...)
key6, value6, server6
>>> GET key7 (expect: key7, value7, server...)
key7, value7, server6
>>> GET key8 (expect: key8, value8, server...)
key8, value8, server6
>>> GET key9 (expect: key9, value9, server...)
key9, value9, server1
>>> GET key10 (expect: key10, value10, server...)
key10, newvalue10, server5
>>> GET key11 (expect: key11, value11, server...)
key11, value11, server6
>>> GET key12 (expect: key12, value12, server...)
key12, value12, server6
>>> GET key13 (expect: key13, value13, server...)
key13, value13, server3
>>> GET key14 (expect: key14, value14, server...)
key14, value14, server6
>>> GET key15 (expect: key15, value15, server...)
key15, newvalue15, server5
>>> GET key16 (expect: key16, value16, server...)
key16, value16, server5
>>> GET key17 (expect: key17, value17, server...)
key17, value17, server4
>>> GET key18 (expect: key18, value18, server...)
key18, value18, server3
>>> GET key19 (expect: key19, value19, server...)
key19, value19, server3
>>> GET key20 (expect: key20, value20, server...)
key20, value20, server1
```